

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Arquitectura de Computadores y Ensambladores 1

Documentación Técnica

Arquitectura, API, MQTT y MongoDB

Proyecto	Estación de Control de la Nave Espacial (Star Wars)
Entregable	Arquitectura, API, MQTT y MongoDB
Equipo	Grupo 5
Fecha	21/03/2026
Observación	El diagrama en Proteus se entrega como anexo independiente provisto por el equipo.

Documento generado a partir del enunciado del proyecto y de la implementación entregada del backend Python + frontend React.

Tabla de contenido

Actualiza la tabla de contenido en Word si deseas numeración dinámica.

1. Objetivo del documento

Este documento describe la arquitectura técnica del proyecto Estación de Control de la Nave Espacial, la integración entre Raspberry Pi, broker MQTT, backend en Python, frontend en React y persistencia en MongoDB. Además documenta los tópicos, endpoints, estructuras de datos y el flujo operativo del sistema.

2. Alcance implementado

- Backend migrado de JavaScript a Python utilizando FastAPI, Socket.IO, PyMongo y paho-mqtt.
- API REST para históricos, estadísticas, salud del sistema, mensajes y comandos.
- Integración asíncrona mediante MQTT para sensores, alertas, actuadores y mensajes.
- Persistencia en MongoDB para lecturas de sensores, eventos, comandos y mensajes.
- Compatibilidad con el dashboard React trabajado por el equipo y con el scaffold original.

3. Arquitectura general

La arquitectura se diseñó para desacoplar la adquisición de datos del hardware, la persistencia y la visualización web. Esto permite que cada subsistema evolucione por separado y facilita la depuración durante las pruebas de integración.

Arquitectura general del sistema

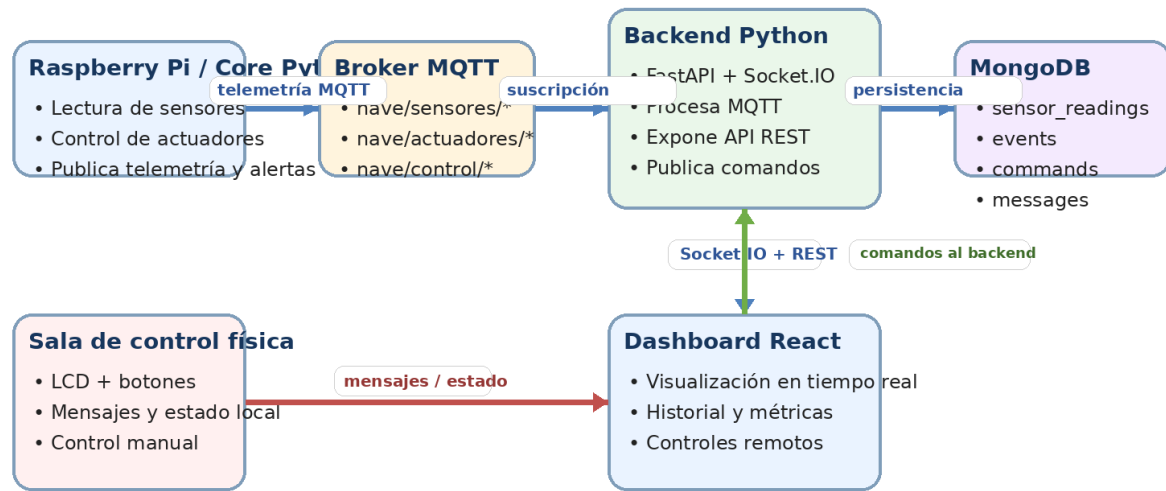


Figura 1. Arquitectura general del sistema.

4. Stack tecnológico

Capa	Tecnología	Responsabilidad	Observaciones
Hardware/Core	Python 3.8+ en Raspberry Pi	Lectura de sensores, control de actuadores y publicación MQTT	Incluye lógica local y sala de control física
Mensajería	Mosquitto MQTT	Comunicación asíncrona entre hardware y backend	QoS 1 para comandos críticos; QoS 0 para telemetría regular
Backend	FastAPI + Socket.IO + paho-mqtt + PyMongo	API REST, tiempo real, traducción de comandos y persistencia	Puede trabajar en modo degradado si faltan servicios externos
Base de datos	MongoDB 5+	Persistencia histórica de lecturas, eventos, comandos y mensajes	Índices por fecha recomendados para consultas
Frontend	React + Vite	Dashboard, controles remotos, métricas y visualización de histórico	Consume REST y Socket.IO

5. Flujo de respuesta automática

El siguiente flujo resume el comportamiento esperado cuando el sistema recibe telemetría, identifica alertas o ejecuta comandos enviados desde el dashboard o desde la sala de control física.

Flujo de respuesta automática

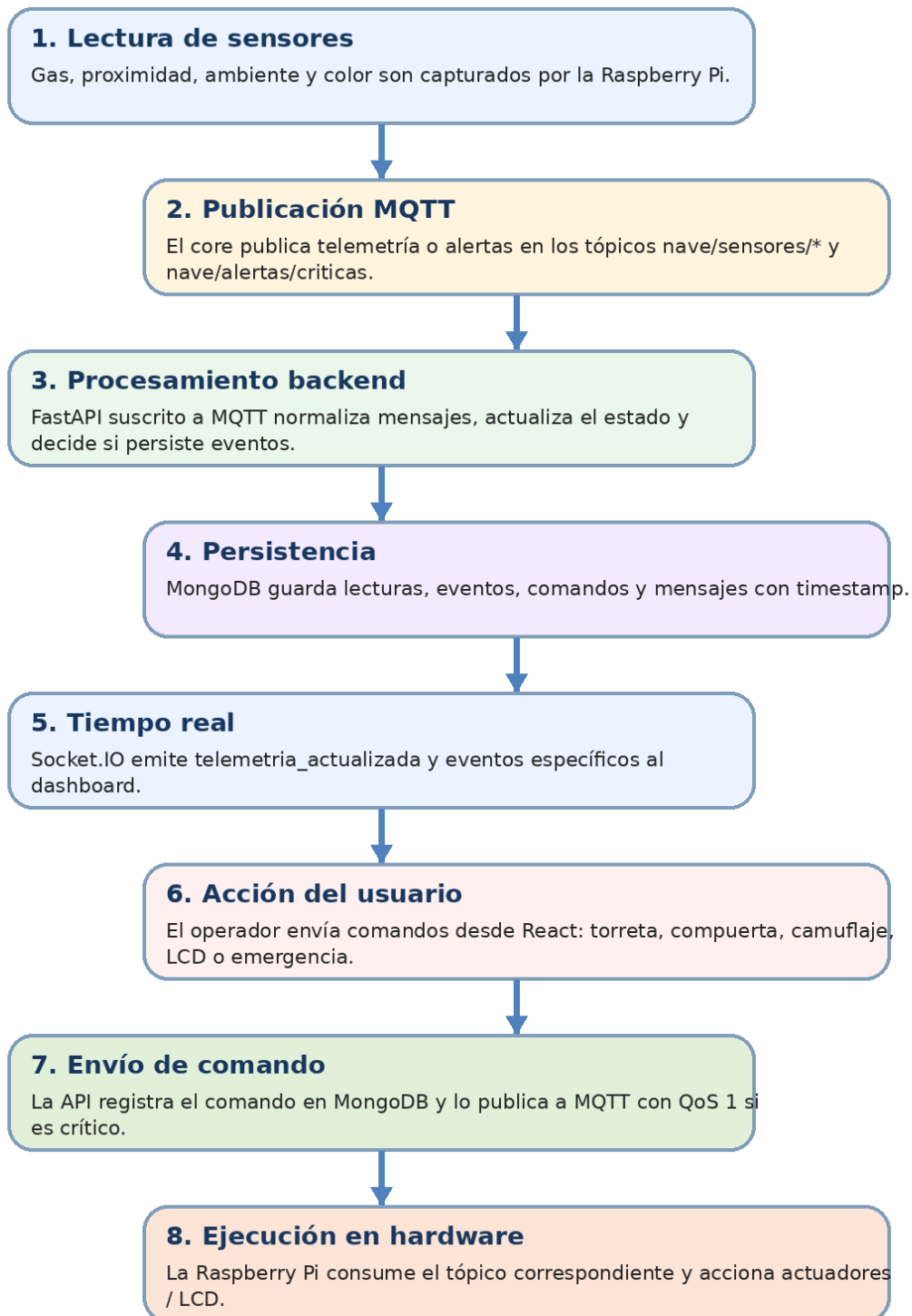


Figura 2. Flujo operativo del procesamiento de telemetría y comandos.

6. Tópicos MQTT

Los tópicos se organizaron por jerarquía funcional para simplificar las suscripciones y la trazabilidad del tráfico durante las pruebas.

Tipo	Topic	Dirección	Uso
Sensor	nave/sensores/gas	RPi -> Backend	Nivel de gas, umbral y estado de peligro
Sensor	nave/sensores/proximidad	RPi -> Backend	Distancia de meteoritos y nivel de alerta
Sensor	nave/sensores/color	RPi -> Backend	Secuencia de colores y estado de camuflaje
Sensor	nave/sensores/ambiente	RPi -> Backend	Temperatura y humedad ambiental
Alerta	nave/alertas/criticas	RPi -> Backend	Eventos severos o emergencias
Estado	nave/estado/general	RPi -> Backend	Estado global de la nave
Actuador	nave/actuadores/torreta	Backend -> RPi	Ángulo, home o disparo
Actuador	nave/actuadores/compuertas	Backend -> RPi	Abrir o cerrar compuertas
Actuador	nave/actuadores/ventiladores	Backend -> RPi	Activación por incendio
Actuador	nave/actuadores/camuflaje	Backend -> RPi	Activación o desactivación
Control	nave/control/emergencia	Backend -> RPi	Detención total y alerta
Control	nave/control/mensajes	Backend -> RPi/LCD	Mensajes de texto para la sala de control

7. Diseño de MongoDB

La base de datos se diseñó para conservar trazabilidad operacional y permitir consultas históricas desde el dashboard.

- **sensor_readings:** Guarda telemetría normalizada con sensorType, value, unit, sourceTopic y timestamp.
- **events:** Registra alertas críticas, sesiones de camuflaje y otros eventos del sistema con severidad y mensaje.
- **commands:** Almacena cada comando publicado por la API, incluyendo topic, payload, QoS y estado.
- **messages:** Persistencia de textos enviados a la LCD o sala de control física.

Índices sugeridos: timestamp en las cuatro colecciones; sensorType + timestamp en sensor_readings; severity + timestamp en events.

8. API REST

La API expone rutas compatibles con el frontend del equipo y también mantiene rutas de compatibilidad con el scaffold original.

Método	Ruta	Propósito	Respuesta resumida
GET	/api/health	Verificar disponibilidad del backend y conexiones	Estado del servicio y modo degradado
GET	/api/sensors/latest	Último estado de sensores y actuadores	Snapshot normalizado
GET	/api/sensores/latest	Último estado adaptado al frontend trabajado	Payload simplificado para el dashboard
GET	/api/sensors/history?type=gas&hours=24	Historial por tipo de sensor	Lista cronológica de lecturas
GET	/api/sensores/historial	Datos agregados para la gráfica de 24 horas	Buckets con gas, meteorito, temperatura y humedad
GET	/api/events	Lista de eventos del sistema	Eventos recientes
GET	/api/messages	Historial de mensajes a LCD	Mensajes con timestamp
POST	/api/messages	Guardar y publicar mensaje a sala de control	Mensaje persistido/publicado
POST	/api/commands	Enviar comando con actuador/action	Comando persistido y topic publicado
POST	/api/comandos	Enviar comando con formato comando/valor	Compatibilidad con frontend trabajado
GET	/api/stats o /api/estadisticas	Métricas agregadas	Disparos, camuflaje, alertas y mensajes

Formato esperado por /api/comandos:

```
{ "comando": "torreta", "valor": 120 }
{ "comando": "compuerta", "valor": "Abrir" }
{ "comando": "camuflaje", "valor": "activar" }
{ "comando": "lcd", "valor": "Mensaje para la sala" }
{ "comando": "emergencia", "valor": true }
```

9. Eventos en tiempo real

- `telemetria_actualizada`: evento principal consumido por el dashboard trabajado.
- `sensor:update`: actualización granular por tipo de sensor.
- `state:update`: cambios del estado general de la nave.
- `actuador:update`: actualización del estado de actuadores.
- `command:sent`, `message:new` y `event:new`: seguimiento operacional detallado.

10. Despliegue y ejecución

- Levantar infraestructura local con `docker compose up -d` dentro de `backend/` para MongoDB y Mosquitto.
- Crear entorno virtual, instalar `requirements.txt`, copiar `.env.example` a `.env` y ejecutar `python run.py`.
- En `frontend/`, instalar dependencias con `npm install`, copiar `.env.example` y ejecutar `npm run dev`.
- Para pruebas sin Raspberry Pi puede utilizarse `demo_publisher.py`, que publica telemetría simulada cada 5 segundos.

11. Pruebas recomendadas

Prueba	Criterio de aceptación
API REST	Probar <code>GET /api/health</code> y <code>POST /api/comandos</code> desde Postman o curl.
MQTT	Confirmar que el backend se suscribe a los tópicos de sensores y que publica comandos a actuadores.
Persistencia	Verificar inserciones en <code>sensor_readings</code> , <code>events</code> , <code>commands</code> y <code>messages</code> .
Dashboard	Confirmar que el estado cambia a ALERTA o EMERGENCIA cuando <code>gas > 50</code> o <code>meteorito < 50</code> cm.
Sala de control	Enviar un mensaje desde el dashboard y comprobar visualización en la LCD.

12. Riesgos y consideraciones

- Mantener una tabla compartida de pines GPIO para evitar conflictos entre sensores, actuadores y LCD.
- Definir payloads MQTT estables y versionados para evitar incompatibilidades entre RPi y backend.
- Monitorear el consumo de corriente cuando se usen ventiladores, LEDs y motores en simultáneo.

- Usar una fuente externa para actuadores y desacoplarla apropiadamente de la Raspberry Pi.

13. Entregables documentales

Junto con este documento deben adjuntarse: el manual de usuario y el diagrama de Proteus del prototipo físico. El plano de Proteus ya fue elaborado por el equipo, por lo que aquí se referencia como anexo externo.